开源软件基础

Python 基础

Zhilei Ren

OSCAR Team, School of Software, Dalian University of Technology

November 23, 2017

Outline

1. 面向对象
2. 函数式特性
   - List Comprehension
   - Map-Reduce
3. 一点更多的数据结构
   - 元组
   - 字典
   - 集合

面向对象

面向对象三要素

- 封装：点号能点出成员
- 继承：儿子点出父亲成员
- 多态：儿子点出和父亲不同的行为

类的声明

```
class ClassName:
    <statement-1>
    .
    .
    .
    <statement-N>
```

__init__ 函数

```
def __init__(self):
    self.data = []
```

Class Objects

```
>>> class Complex:
...     def __init__(self, realpart, imagpart):
...         self.r = realpart
...         self.i = imagpart
...
>>> x = Complex(3.0, -4.5)
>>> x.r, x.i
(3.0, -4.5)
```

成员变量和实例变量

```
class Dog:

    tricks = []             # mistaken use of a class

    def __init__(self, name):
        self.name = name

    def add_trick(self, trick):
        self.tricks.append(trick)

>>> d = Dog('Fido')
>>> e = Dog('Buddy')
>>> d.add_trick('roll over')
```

Class Objects

```
class MyClass:
    """A simple example class"""
    i = 12345

    def f(self):
        return 'hello world'
```

实例/类的成员

- 在 __init__ 中用 self. 初始化的是实例成员
- 在类似 C++ 成员声明处初始化的是类成员（类似静态变量）
- 实例成员可以动态添加和删除
- 必要时用 hasattr 判断是否包含某成员

成员可见性

成员可见性

- 不存在类似 C++ 的私有
- 约定：下划线前缀的是实现细节
- ipython/Jupyter 默认不会补全
- 类似 *nix 下的 dotfiles

继承

```
class A:
    def foo(self):
        print("in A::foo")

class B(A):
    pass

b = B()
b.foo()
```

多态

```python
class A:
    def foo(self):
        print("in A::foo")

class B(A):
    def foo(self):
        print("in B::foo")

b = B()
b.foo()
isinstance(b, B)
isinstance(b, A)
```

鸭子类型

在程序设计中，鸭子类型（英语：duck typing）是动态类型的一种风格。在这种风格中，一个对象有效的语义，不是由继承自特定的类或实现特定的接口，而是由" 当前方法和属性的集合" 决定。这个概念的名字来源于由 James Whitcomb Riley 提出的鸭子测试，“鸭子测试”可以这样表述：

鸭子测试

- If it walks like a duck
- It quacks like a duck
- Then it must be a duck

“当看到一只鸟走起来像鸭子、游泳起来像鸭子、叫起来也像鸭子，那么这只鸟就可以被称为鸭子。”[1]

[1] https://en.wikipedia.org/wiki/Duck_typing

多态

```
class A:
    def foo(self):
        print("in A::foo")

class B(A):
    def foo(self):
        print("in B::foo")

class C:
    def foo(self):
        print("not subclass of A/B")

def test_class(a):
    a.foo()
```

Outline

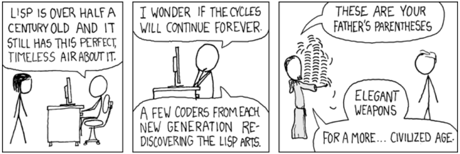
1. 面向对象
2. 函数式特性
   - List Comprehension
   - Map-Reduce
3. 一点更多的数据结构
   - 元组
   - 字典
   - 集合

函数一等公民

- 可以被存入变数或其他结构
- 可以被作为参数传递给其他函数
- 可以被作为函数的返回值
- 可以在执行期创造，而无需完全在设计期全部写出
- 即使没有被系结至某一名称，也可以存在

Lisp

```
(+ 1 2)
(define (foo bar) (+ bar 1))
(foo 100)
(define (fib x)
    (if (<= x 1)
        1
        (+ (fib (- x 1))
             (fib (- x 2)))))
(display (fib 10))
```



Haskell

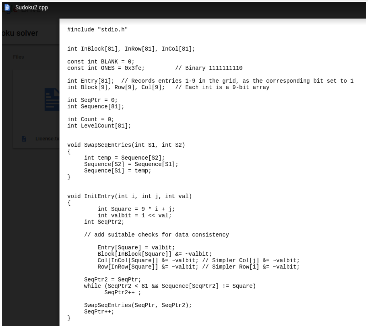
- 偏学术
- 号称最纯粹的函数式语言
- Avoid Success at all Cost

今天就能开始用的小知识

关于 Haskell

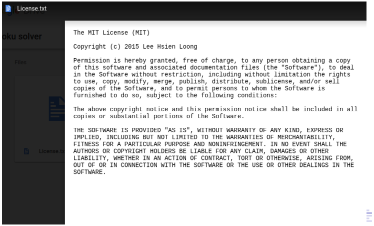
- Avoid Success at all Cost, or Avoid "Success at all cost"
- 新加坡总理李显龙（会 C/C++）称退休后会学习 Haskell [a]

[a] http://www.pmo.gov.sg/newsroom/transcript-speech-prime-minister-lee-hsien-loong-founders-forum-s



Sudoku2.cpp

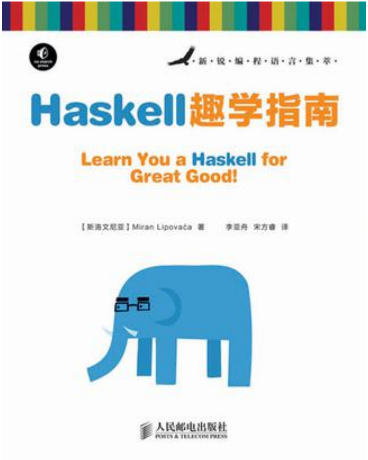
```
#include "stdio.h"


int InBlock[81], InRow[81], InCol[81];

const int BLANK = 0;
const int ONES = 0x3fe;        // Binary 1111111110

int Entry[81];  // Records entries 1-9 in the grid, as the corresponding bit set to 1
int Block[9], Row[9], Col[9];   // Each int is a 9-bit array

int SeqPtr = 0;
int Sequence[81];

int Count = 0;
int LevelCount[81];


void SwapSeqEntries(int S1, int S2)
{
     int temp = Sequence[S2];
     Sequence[S2] = Sequence[S1];
     Sequence[S1] = temp;
}


void InitEntry(int i, int j, int val)
{
         int Square = 9 * i + j;
         int valbit = 1 << val;
     int SeqPtr2;

     // add suitable checks for data consistency

         Entry[Square] = valbit;
         Block[InBlock[Square]] &= ~valbit;
         Col[InCol[Square]] &= ~valbit; // Simpler Col[j] &= ~valbit;
         Row[InRow[Square]] &= ~valbit; // Simpler Row[i] &= ~valbit;

     SeqPtr2 = SeqPtr;
     while (SeqPtr2 < 81 && Sequence[SeqPtr2] != Square)
           SeqPtr2++ ;

     SwapSeqEntries(SeqPtr, SeqPtr2);
     SeqPtr++;
}
```



License.txt

```
The MIT License (MIT)

Copyright (c) 2015 Lee Hsien Loong

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

书籍推荐


·新·锐·编·程·语·言·集·萃·
Haskell趣学指南
Learn You a Haskell for Great Good!
【斯洛文尼亚】Miran Lipovača 著 李亚舟 宋方睿 译
人民邮电出版社
POSTS & TELECOM PRESS

什么是函数式

函数式编程（functional programming）或称函数程序设计，是一种编程典范，它将电脑运算视为数学上的函数计算，并且避免使用程序状态以及易变对象。函数编程语言最重要的基础是 λ 演算（lambda calculus）。而且 λ 演算的函数可以接受函数当作输入（引数）和输出（传出值）。
比起命令式编程，函数式编程更加强调程序执行的结果而非执行的过程，倡导利用若干简单的执行单元让计算结果不断渐进，逐层推导复杂的运算，而不是设计一个复杂的执行过程。

函数式特性

- First-class and higher-order functions
- Pure function
- Recursion
- Type system
- ...

λ 演算

λ 演算（英语：lambda calculus，λ-calculus）是一套从数学逻辑中发展，以变量绑定和替换的规则，来研究函数如何抽象化定义、函数如何被应用以及递归的形式系统。它由数学家阿隆佐·邱奇在 20 世纪 30 年代首次发表。Lambda 演算作为一种广泛用途的计算模型，可以清晰地定义什么是一个可计算函数，而任何可计算函数都能以这种形式表达和求值，它能模拟单一磁带图灵机的计算过程；尽管如此，Lambda 演算强调的是变换规则的运用，而非实现它们的具体机器。

λ 表达式

```
inc = lambda x: x + 1

add = lambda x, y: x + y

inc(5)
add(10, 20)

# 注意：Python的lambda表达式不能换行
```

Outline

列表推导

```
[i * 2 for i in range(10)]

[i * 2 for i in range(10) if i % 2 == 0]

colours = ["red", "green", "yellow", "blue"]
things = ["house", "car", "tree"]
combined =  [(x,y) for x in colours for y in things]
```

Outline

Map/Filter/Reduce

```
map(lambda x: x ** 2, range(10)
list(map(lambda x: x **2, range(10))

filter(lambda x: x % 2 == 0,
        map(lambda x: x ** 2, range(10)))

reduce(lambda x, y: x + y,
        filter(lambda x: x % 2 == 0,
                map(lambda x: x ** 2, range(10))))
```

Outline

1. 面向对象
2. 函数式特性
   - List Comprehension
   - Map-Reduce
3. 一点更多的数据结构
   - 元组
   - 字典
   - 集合

Outline

1. 面向对象
2. 函数式特性
   - List Comprehension
   - Map-Reduce
3. 一点更多的数据结构
   - 元组
   - 字典
   - 集合

Tuples and Sequences

- 元组与列表类似，都是线性数据结构
- 元组由逗号分割的数据构成
- 元组内容不可变
- 元组可以嵌套

Tuples and Sequences

```
>>> t = 12345, 54321, 'hello!'
>>> t[0]
12345
>>> t
(12345, 54321, 'hello!')
>>> # Tuples may be nested:
... u = t, (1, 2, 3, 4, 5)
>>> u
((12345, 54321, 'hello!'), (1, 2, 3, 4, 5))
>>> # Tuples are immutable:
... t[0] = 88888
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
>>> # but they can contain mutable objects:
... v = ([1, 2, 3], [3, 2, 1])
>>> v
([1, 2, 3], [3, 2, 1])
```

Tuples and Sequences

- 构造空列表使用 ()
- 构造单元素列表用 (a,)
- 如果元组元素是可变的，则可以修改（有坑）
- 元组也可出现在赋值左部（lhs）

Tuples and Sequences

```
>>> empty = ()
>>> singleton = 'hello',    # <-- note trailing comma
>>> len(empty)
0
>>> len(singleton)
1
>>> singleton
('hello',)
```

Tuples and Sequences

```
>>> x, y, z = t
>>> x, y = y, x

>>> x = (1, 2, 3, [4, 5])
>>> x[3] += [6]
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-34-1b7fadc4e686> in <module>()
----> 1 x[3] += [6]

TypeError: 'tuple' object does not support item assignment

>>> x
(1, 2, 3, [4, 5, 6])
```

Outline

Dictionaries

- 字典类似其他语言中散列表、关联数组，按键值方式组织
- 不像列表元组可以用下标随机访问
- 任何不可变对象均可作为键（元组亦可，但不能包含可变元素）
- 空字典用 {} 构造；非空字典在 {} 内逗号分割的键: 值对表示
- list(d.keys()) 返回所有键。如需有序，使用 sorted(d.keys())

Dictionaries

```
>>> tel = {'jack': 4098, 'sape': 4139}
>>> tel['guido'] = 4127
>>> tel
{'sape': 4139, 'guido': 4127, 'jack': 4098}
>>> tel['jack']
4098
>>> del tel['sape']
>>> tel['irv'] = 4127
>>> tel
{'guido': 4127, 'irv': 4127, 'jack': 4098}
>>> list(tel.keys())
['irv', 'guido', 'jack']
>>> sorted(tel.keys())
['guido', 'irv', 'jack']
>>> 'guido' in tel
True
>>> 'jack' not in tel
False
```

Dictionaries

```
>>> x = {}
>>> x[(1, 2, 3)] = 4
>>> x[(1, 2, [3, 4])] = 5
>>> x[(1, 2, 3, [4, 5])] = 6
---------------------------------------------------------------------------
TypeError         Traceback (most recent call last)
<ipython-input-8-7e134042fef1> in <module>()
----> 1 x[(1, 2, 3, [4, 5])] = 6

TypeError: unhashable type: 'list'
```

Dictionaries

- 字典还可以由二元组列表初始化
- 字典推导与列表推导类似
- 字典键为字符串时，可以用关键字参数设定

Dictionaries

```
>>> dict([('sape', 4139),
          ('guido', 4127),
          ('jack', 4098)])
{'sape': 4139, 'jack': 4098, 'guido': 4127}
```

Dictionaries

```
>>> {x: x**2 for x in (2, 4, 6)}
{2: 4, 4: 16, 6: 36}
```

Dictionaries

```
>>> dict(sape=4139, guido=4127, jack=4098)
{'sape': 4139, 'jack': 4098, 'guido': 4127}
```

Outline

1. 面向对象
2. 函数式特性
   - List Comprehension
   - Map-Reduce
3. 一点更多的数据结构
   - 元组
   - 字典
   - 集合

Sets

- 集合是无序无重复的数据结构
- 支持成员测试、添加、删除等操作
- 支持交集、并集、差等集合操作
- 初始化用 {} 括住的逗号分割元素序列
- 空集合用 set()，而不是 {}
- 集合推导同样支持

Sets

```
>>> basket = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}
>>> print(basket)                      # show that duplicates have been re
{'orange', 'banana', 'pear', 'apple'}
>>> 'orange' in basket                 # fast membership testing
True
>>> 'crabgrass' in basket
False

>>> # Demonstrate set operations on unique letters from two words
...
>>> a = set('abracadabra')
>>> b = set('alacazam')
>>> a                                  # unique letters in a
{'a', 'r', 'b', 'c', 'd'}
>>> a - b                              # letters in a but not in b
{'r', 'd', 'b'}
>>> a | b                              # letters in a or b or both
{'a', 'c', 'r', 'd', 'b', 'm', 'z', 'l'}
>>> a & b                              # letters in both a and b
{'a', 'c'}
>>> a ^ b                              # letters in a or b but not both
{'r', 'd', 'b', 'm', 'z', 'l'}
```

Sets

```
>>> a = {x for x in 'abracadabra' if x not in 'abc'}
>>> a
{'r', 'd'}
```